L40 — Heaps: Min-Heap and Max-Heap

Unit: 3 | **Chapter:** Trees | **BT Level:** BT5 | **CO:** CO2, CO3

Learning Objectives

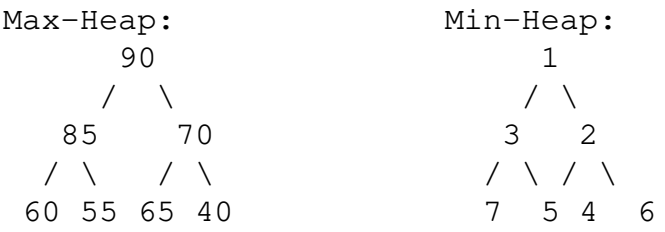
- Define the heap property for min-heaps and max-heaps
- Implement heap operations: insert, extract, heapify
- Represent a heap as an array and navigate using index formulas
- Build a heap from an arbitrary array in O(n) time

---

1. Heap Definition

A **heap** is a complete binary tree satisfying the **heap property**:

- **Max-Heap:** Every parent ≥ both children → root is the maximum.
- **Min-Heap:** Every parent ≤ both children → root is the minimum.



Key property: The root is always the max (or min), but siblings have no ordering relationship.

---

2. Array Representation

Heap nodes are stored level by level in an array (1-indexed):

```
Array: [_, 90, 85, 70, 60, 55, 65, 40]
Index:      1   2   3   4   5   6   7
```

For node at index i:

- **Parent:** `i // 2`
- **Left child:** `2 * i`
- **Right child:** `2 * i + 1`

(0-indexed: parent = `(i-1)//2`, left = `2i+1`, right = `2i+2`)

---

3. Max-Heap Implementation

```
class MaxHeap:
```

```
def __init__(self):
    self.heap = []   # 0-indexed

def _parent(self, i): return (i - 1) // 2
def _left(self, i): return 2 * i + 1
def _right(self, i): return 2 * i + 2

def _swap(self, i, j):
    self.heap[i], self.heap[j] = self.heap[j], self.heap[i]

def peek(self):
    """Return max without removing. O(1)"""
    if not self.heap:
        raise IndexError("Heap is empty")
    return self.heap[0]

def insert(self, val):
    """Insert value and restore heap property. O(log n)"""
    self.heap.append(val)
    self._sift_up(len(self.heap) - 1)

def _sift_up(self, i):
    """Bubble element up until heap property restored."""
    while i > 0:
        parent = self._parent(i)
        if self.heap[i] > self.heap[parent]:
            self._swap(i, parent)
            i = parent
        else:
            break

def extract_max(self):
    """Remove and return max element. O(log n)"""
    if not self.heap:
        raise IndexError("Heap is empty")
    if len(self.heap) == 1:
        return self.heap.pop()

    # Swap root with last, remove last, sift down root
    max_val = self.heap[0]
    self.heap[0] = self.heap.pop()   # Move last element to root
    self._sift_down(0)
    return max_val

def _sift_down(self, i):
    """Push element down until heap property restored."""
    n = len(self.heap)
    while True:
        largest = i
        left = self._left(i)
        right = self._right(i)

        if left < n and self.heap[left] > self.heap[largest]:
```

```
            largest = left
        if right < n and self.heap[right] > self.heap[largest]:
            largest = right

        if largest != i:
            self._swap(i, largest)
            i = largest
        else:
            break
```

---

4. Insertion Trace

Insert 80 into Max-Heap: [90, 85, 70, 60, 55, 65, 40]

1. Append 80 at index 7: `[90, 85, 70, 60, 55, 65, 40, 80]`
2. Sift up: index 7, parent = 3 (value 60); 80 > 60 → swap: `[90, 85, 70, 80, 55, 65, 40, 60]`
3. Sift up: index 3, parent = 1 (value 85); 80 < 85 → stop.

Final: `[90, 85, 70, 80, 55, 65, 40, 60]` ✓

---

5. Extract-Max Trace

Extract max from [90, 85, 70, 80, 55, 65, 40, 60]:

1. Save max = 90
2. Move last (60) to root: `[60, 85, 70, 80, 55, 65, 40]`
3. Sift down from index 0:
    - Compare 60 with children: left = 85 (idx 1), right = 70 (idx 2). Largest = 85 → swap: `[85, 60, 70, 80, 55, 65, 40]`
    - At index 1: left = 80 (idx 3), right = 55 (idx 4). Largest = 80 → swap: `[85, 80, 70, 60, 55, 65, 40]`
    - At index 3: children are at idx 7,8 — don't exist. Stop.

Final: `[85, 80, 70, 60, 55, 65, 40]` ✓ (85 is new max)

---

6. Build Heap from Array — O(n)

Naive approach: Insert n elements one by one → O(n log n).

Optimal approach (Heapify): Start from last non-leaf, sift down each node.

```
def build_max_heap(arr):
    """Build max-heap in-place in O(n) time."""
    n = len(arr)

    def sift_down(heap, i, size):
        while True:
```

```
            largest = i
            left, right = 2*i+1, 2*i+2
            if left < size and heap[left] > heap[largest]:
                largest = left
            if right < size and heap[right] > heap[largest]:
                largest = right
            if largest != i:
                heap[i], heap[largest] = heap[largest], heap[i]
                i = largest
            else:
                break

    # Start from last internal node
    for i in range(n // 2 - 1, -1, -1):
        sift_down(arr, i, n)

    return arr

arr = [4, 10, 3, 5, 1, 9]
print(build_max_heap(arr))   # [10, 5, 9, 4, 1, 3]
```

Why O(n)?
Most nodes are near the bottom and require little sifting. The cost is: $$\sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} \cdot h = O(n)$$

---

7. Min-Heap (via heapq)

Python's `heapq` module implements a **min-heap**:

```
import heapq

heap = []
heapq.heappush(heap, 5)
heapq.heappush(heap, 3)
heapq.heappush(heap, 7)
heapq.heappush(heap, 1)

print(heap)                  # [1, 3, 7, 5] (min-heap)
print(heapq.heappop(heap))   # 1 (min)
print(heapq.heappop(heap))   # 3

# Build heap from list
arr = [4, 10, 3, 5, 1, 9]
heapq.heapify(arr)           # O(n) — converts in place
print(arr)                   # [1, 4, 3, 5, 10, 9]
```

---

8. Complexity Summary

Operation	Time	Notes
peek (min/max)	O(1)	Root access

insert	O(log n)	Sift up at most h levels
extract-min/max	O(log n)	Sift down at most h levels
build heap	O(n)	Linear time heapify
search	O(n)	Heap has no ordering except root
delete (arbitrary)	O(log n)	Replace with last, sift

Summary

- **Heap:** complete binary tree with parent $\geq$ children (max-heap) or parent $\leq$ children (min-heap).
- Array representation: parent at i, children at $2i+1$ and $2i+2$ (0-indexed).
- **Insert:** append, sift up — O(log n).
- **Extract:** swap root with last, remove, sift down — O(log n).
- **Build heap:** start from last internal node, sift down — O(n).

References

- Cormen et al., *Introduction to Algorithms*, Ch. 6 (MIT Press, 2022)
- Python `heapq` module documentation